Machine Learning

Support Vector Machine, Final

Dong-Kyu Chae

PI of the Data Intelligence Lab @HYU
Department of Computer Science
Hanyang University

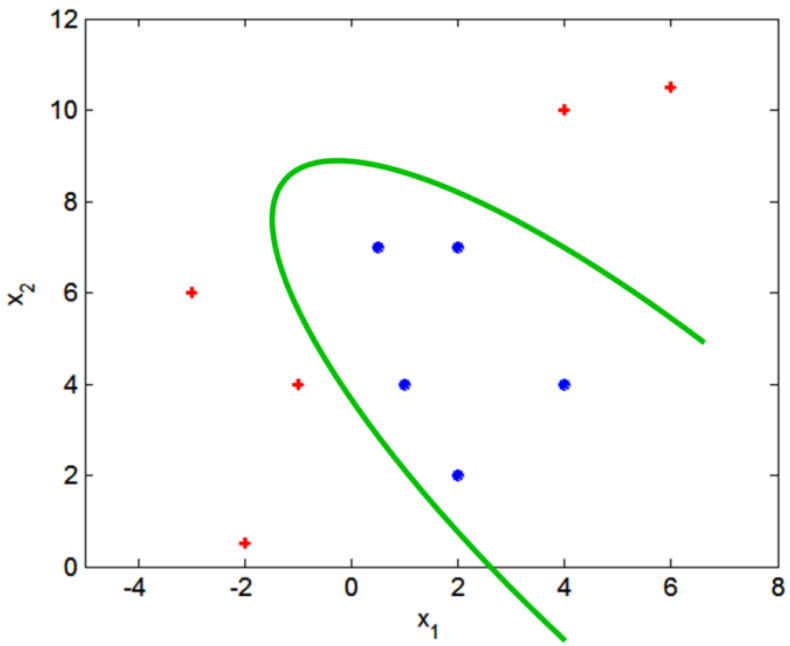
Data
Intelligence
LAB

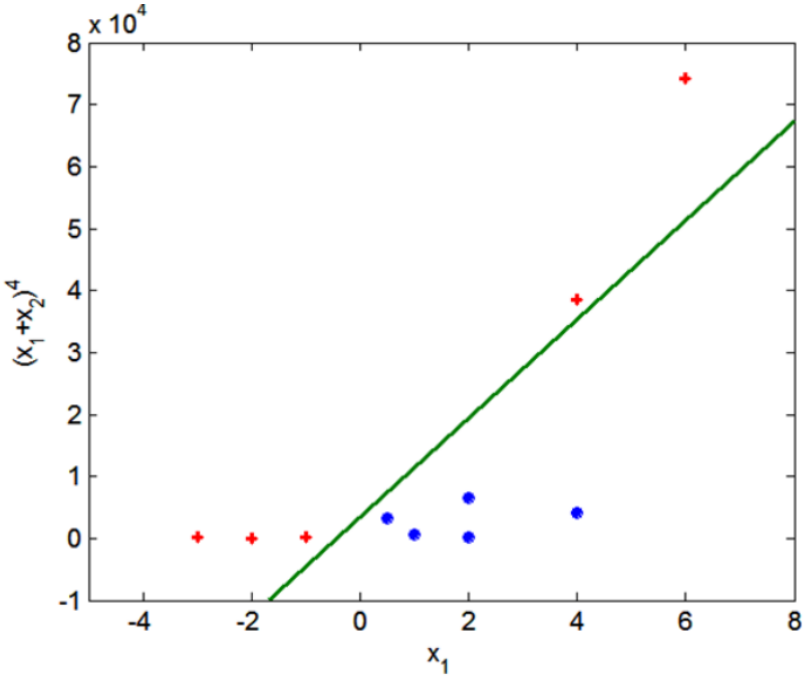
Contents

- **What is SVM ?**
- **Objective function of SVM**
- **Optimization with constraints**
  - Lagrange multiplier
  - Application to SVM
- **Soft margin SVM**
- **Kernel trick**
  - Application to SVM

Non-linear SVM

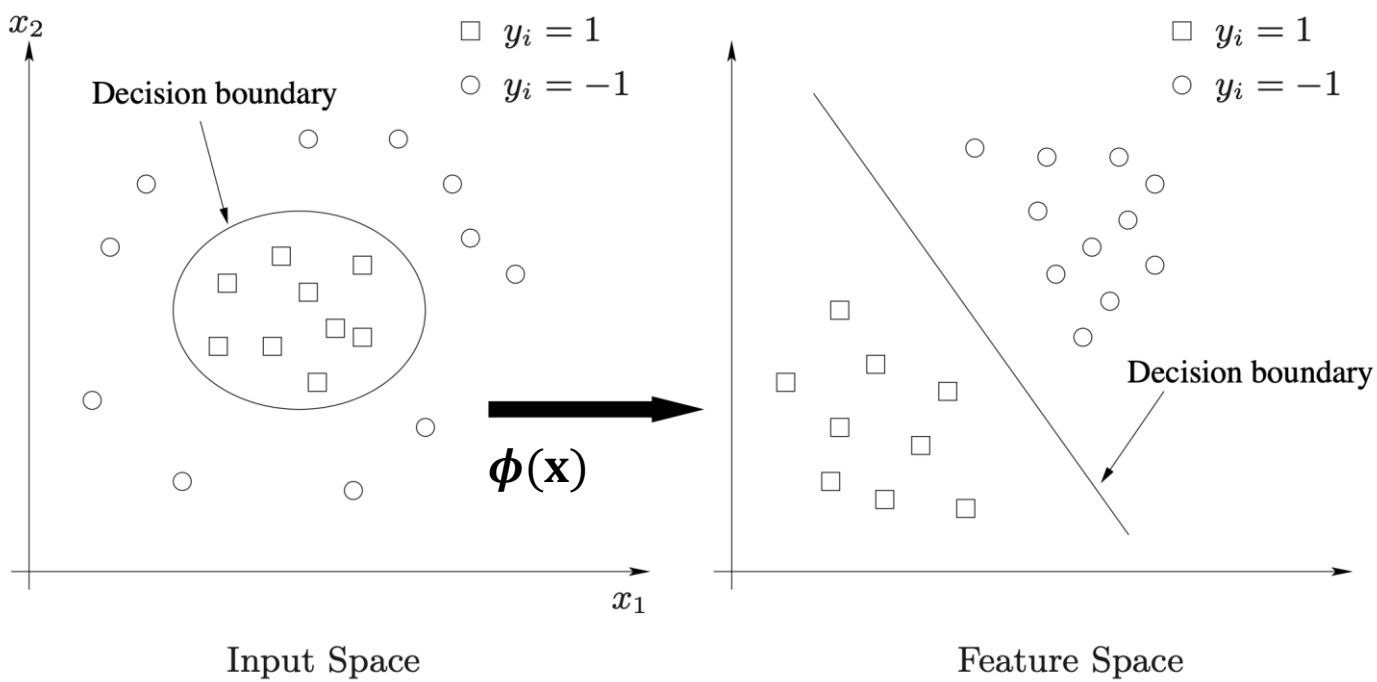
- **Option 2: Non-linearity!**
  - Drawing non-linear decision boundary
  - Which is, equivalent to transforming input vectors into new space, and then draw a linear decision boundary on the new space





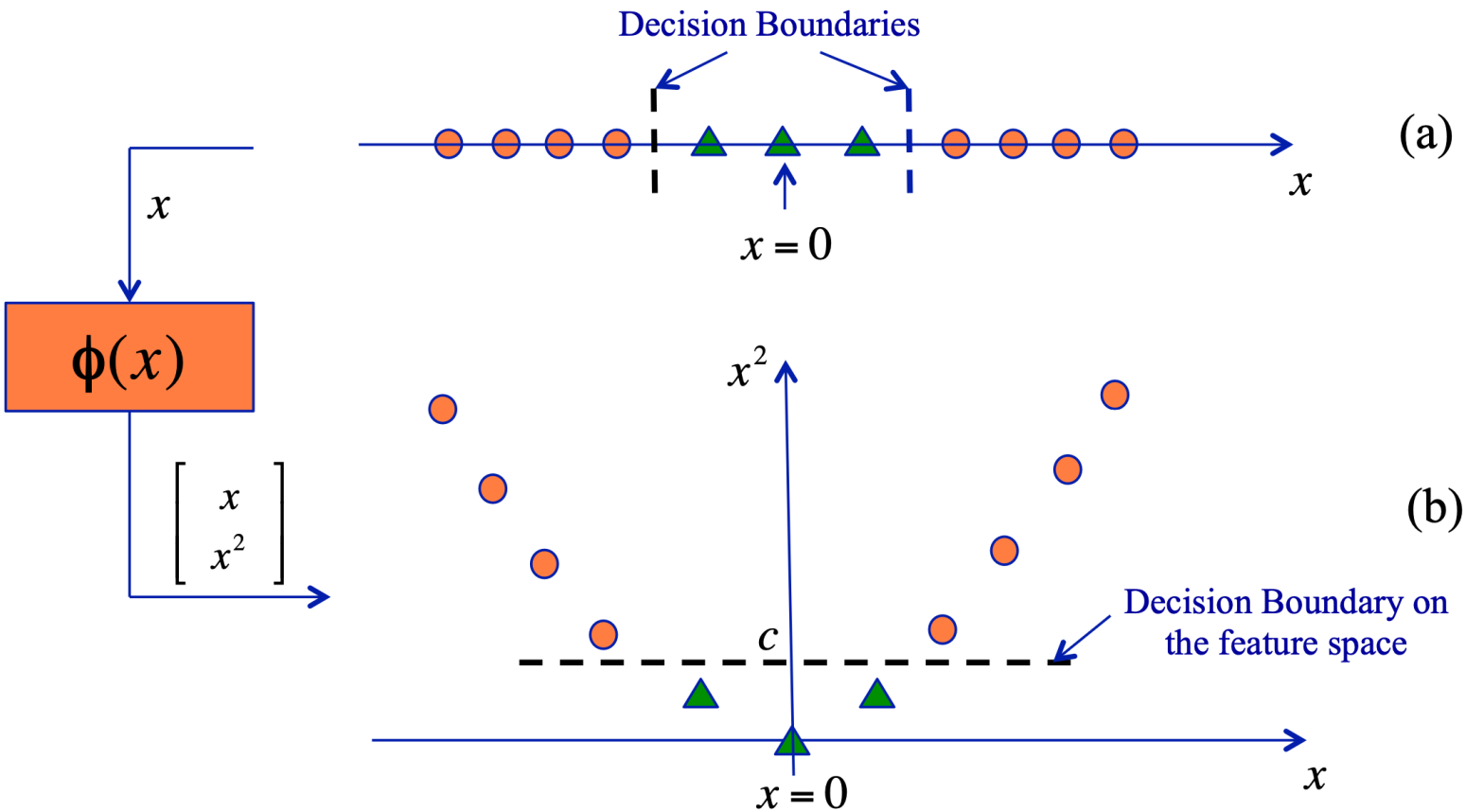
Transformation to High-Dimensional Space

- Assume that we have a nonlinear function $\boldsymbol{\phi}(\mathbf{x})$ that map a data **x** from the input space to a much higher (possibly infinite) dimensional space called the **feature space**.
- While data are not linearly separable in the input space, they will become linearly separable in the feature space.



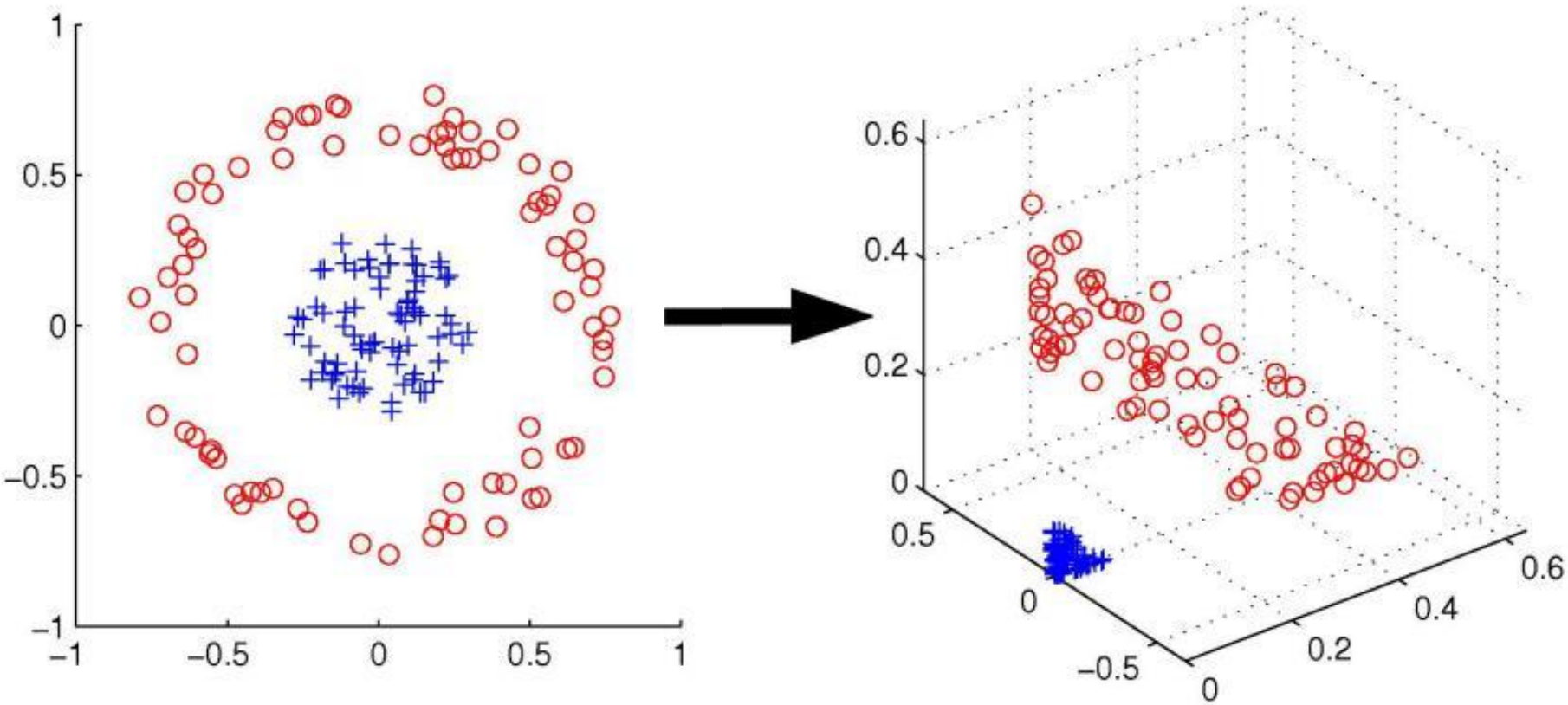
Transformation to High-Dimensional Space

- **A 1-D problem requiring a non-linear decision boundary**
  - A linear SVM may not be able to solve this problem
- **How about transformation via $\phi(\mathbf{x}) = \begin{bmatrix} x \\ x^2 \end{bmatrix}$?**



Transformation to High-Dimensional Space

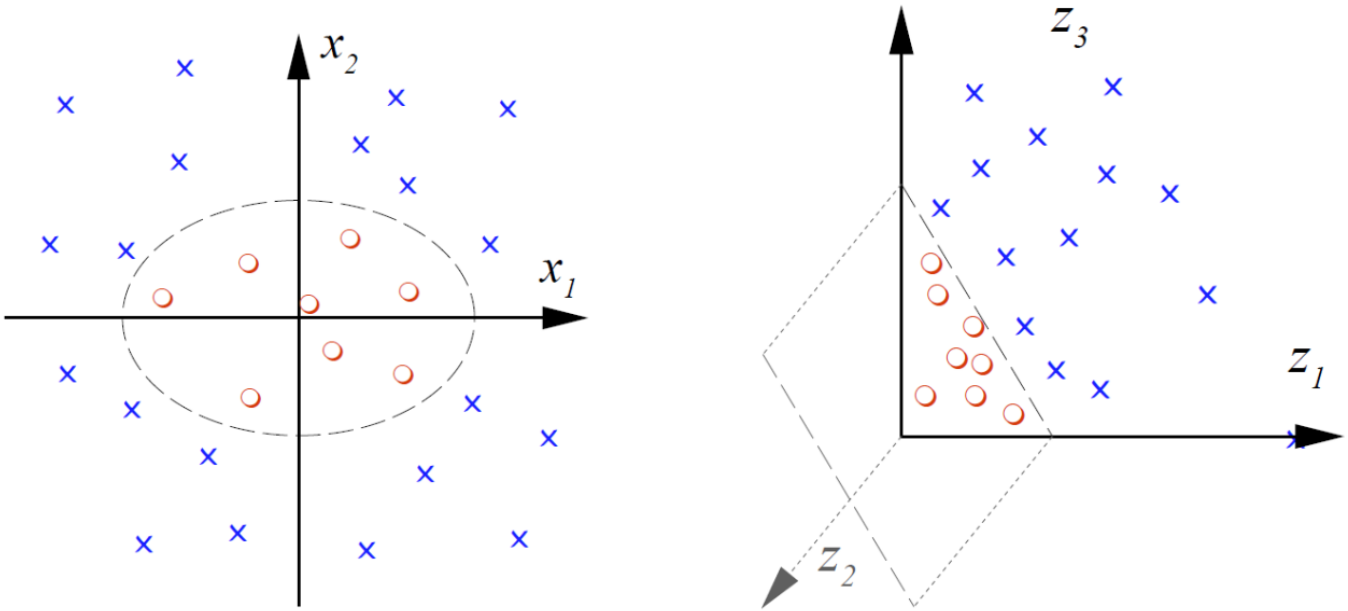
- **Make it linearly separable by transformation of** $\Phi\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \begin{bmatrix} (x_1)^2 \\ (x_2)^2 \\ \sqrt{2}x_1x_2 \end{bmatrix}$



Transformation to High-Dimensional Space

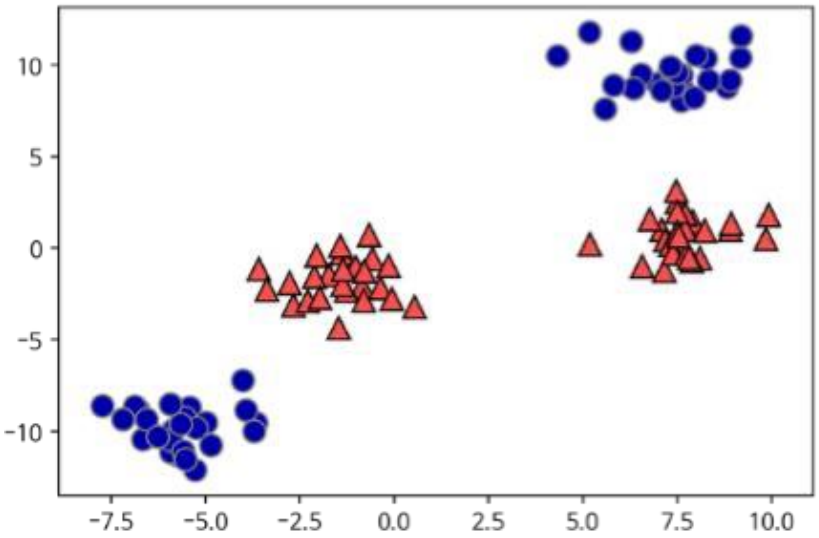
- **Generally, more likely to be linearly separable in higher dimension!**

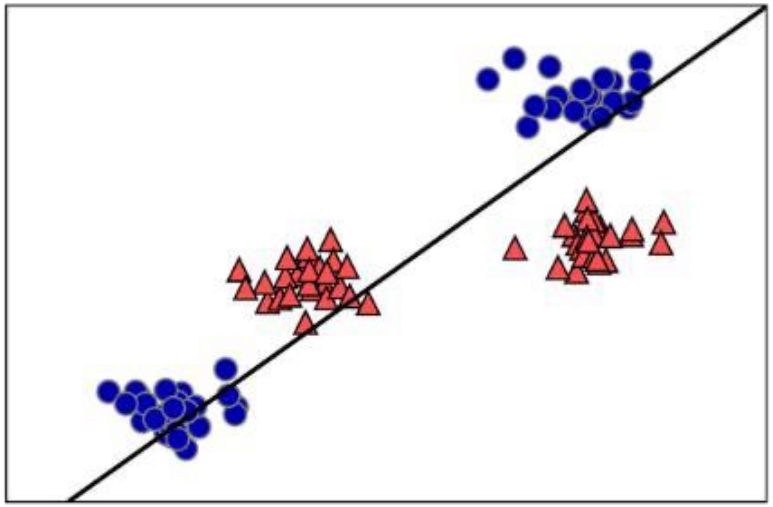
$$\Phi : \mathbb{R}^2 \to \mathbb{R}^3$$
$$(x_1, x_2) \mapsto (z_1, z_2, z_3) := (x_1^2, \sqrt{2}\, x_1 x_2, x_2^2)$$

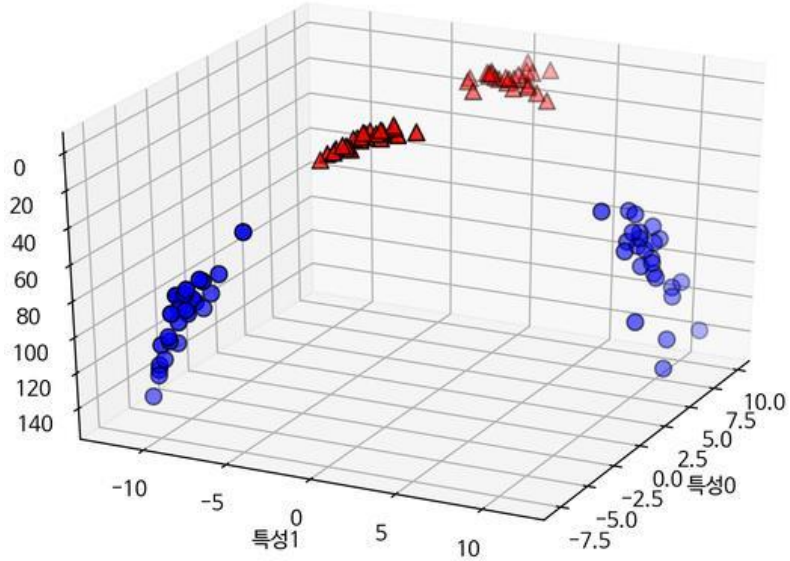


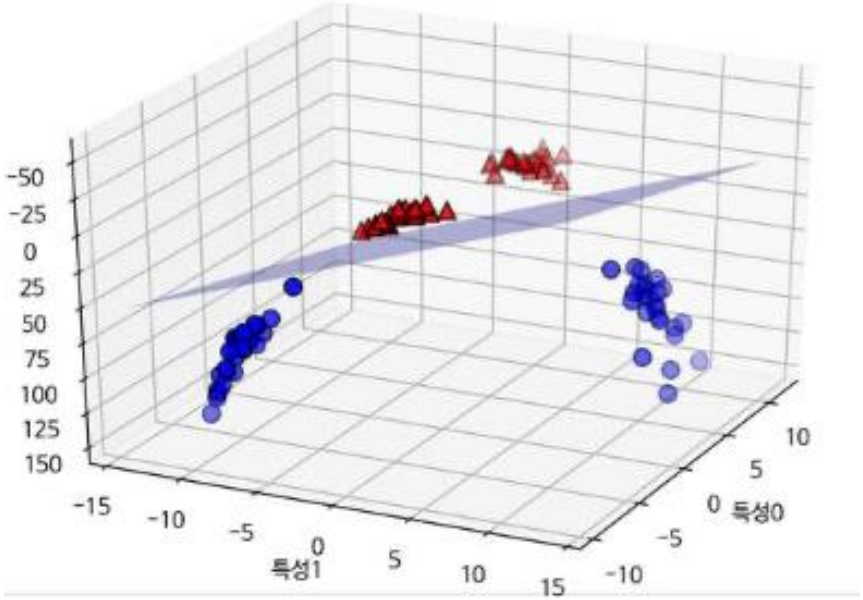
$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \qquad \mathbf{z} = \Phi(\mathbf{x}) = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \begin{bmatrix} x_1^2 \\ \sqrt{2} x_1 x_2 \\ x_2^2 \end{bmatrix}$$

Transformation to High-Dimensional Space



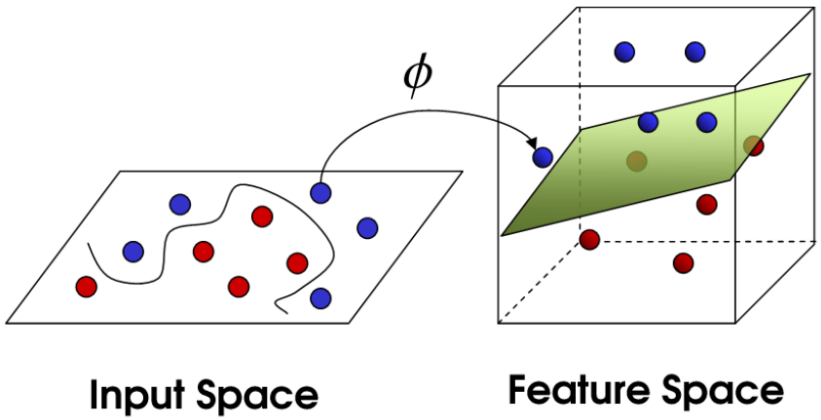


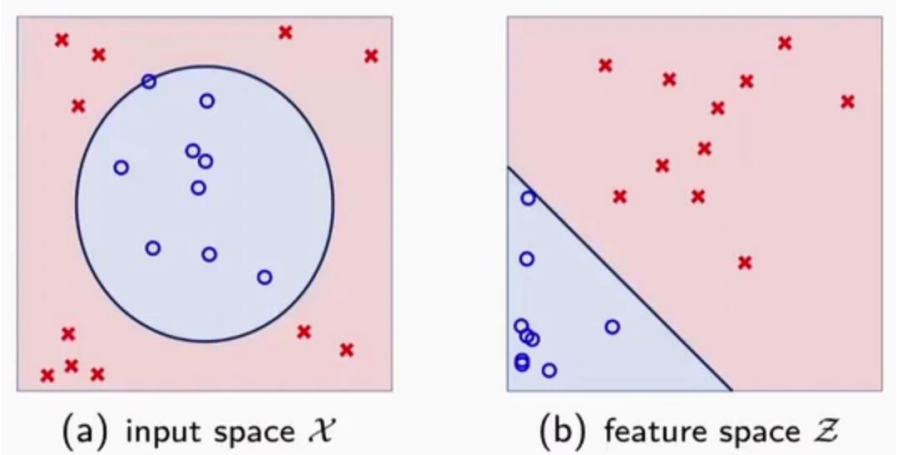




SVM on the Feature Space: Non-linear SVM

- **A (non-linear) transformation function $\Phi(\mathbf{x})$**
  - $\Phi: \mathcal{X} \rightarrow \mathcal{Z}$ where $\mathcal{X}$ is **input space** and $\mathcal{Z}$ is **feature space**
  - $\Phi(\mathbf{x}) = \boldsymbol{z} \in \mathcal{Z}$
  - Linearly non-separable data in $\mathcal{X}$ may become separable in $\mathcal{Z}$





- **Apply SVM to that feature space, instead of input space!**

SVM on the Feature Space : Non-linear SVM

- **Objective function: (not changed)**

$$\min \quad \frac{1}{2}||\mathbf{w}||^2 + C\sum_{i=1}^{N}\xi_i$$

- **Modified constraints:**

$$s.t. \quad y_i(\mathbf{w}^T\boxed{\Phi(\mathbf{x}_i)} + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i$$

- **Lagrange Problem**

$$\min \quad L_P(\mathbf{w}, b, \alpha_i) = \frac{1}{2}||\mathbf{w}||^2 + C\sum_{i=1}^{n}\xi_i$$

$$- \sum_{i=1}^{n}\alpha_i\big(y_i(\mathbf{w}^T\boxed{\Phi(\mathbf{x}_i)} + b) - 1 + \xi_i\big) - \sum_{i=1}^{n}\mu_i\xi_i$$

SVM on the Feature Space: Non-linear SVM

Solutions of the primal problem

$$\frac{\partial L_P}{\partial \mathbf{w}} = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \boxed{\Phi(\mathbf{x}_i)} \qquad \frac{\partial L_P}{\partial b} = 0 \Rightarrow \sum_{i=1}^{n} \alpha_i y_i = 0$$

$$\frac{\partial L_P}{\partial \xi_i} = 0 \Rightarrow C - \alpha_i - \mu_i = 0$$

Dual form

$$\max \quad L_D = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \boxed{\Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j)}$$

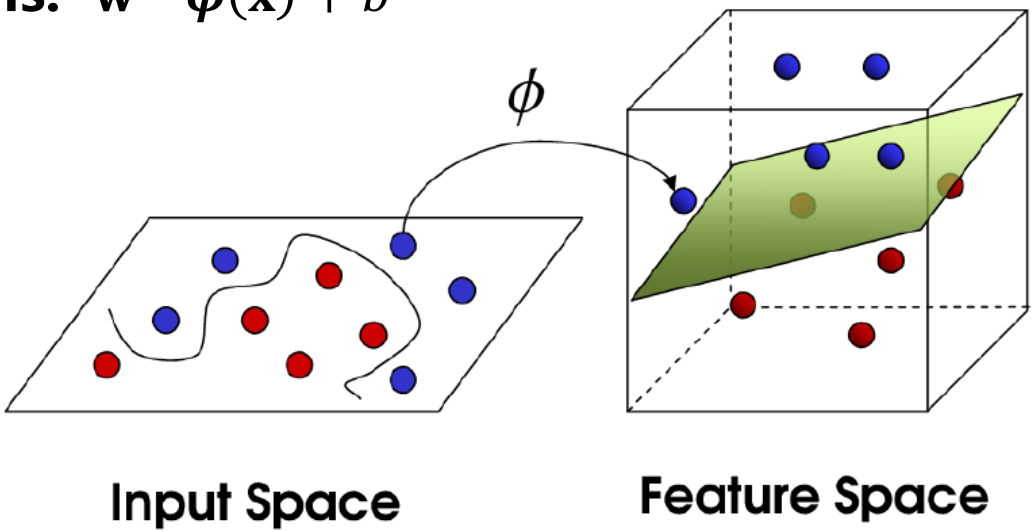
$$s.t. \quad \sum_{i=1}^{n} \alpha_i y_i = 0 \quad and \quad 0 \leq \alpha_i \leq C$$

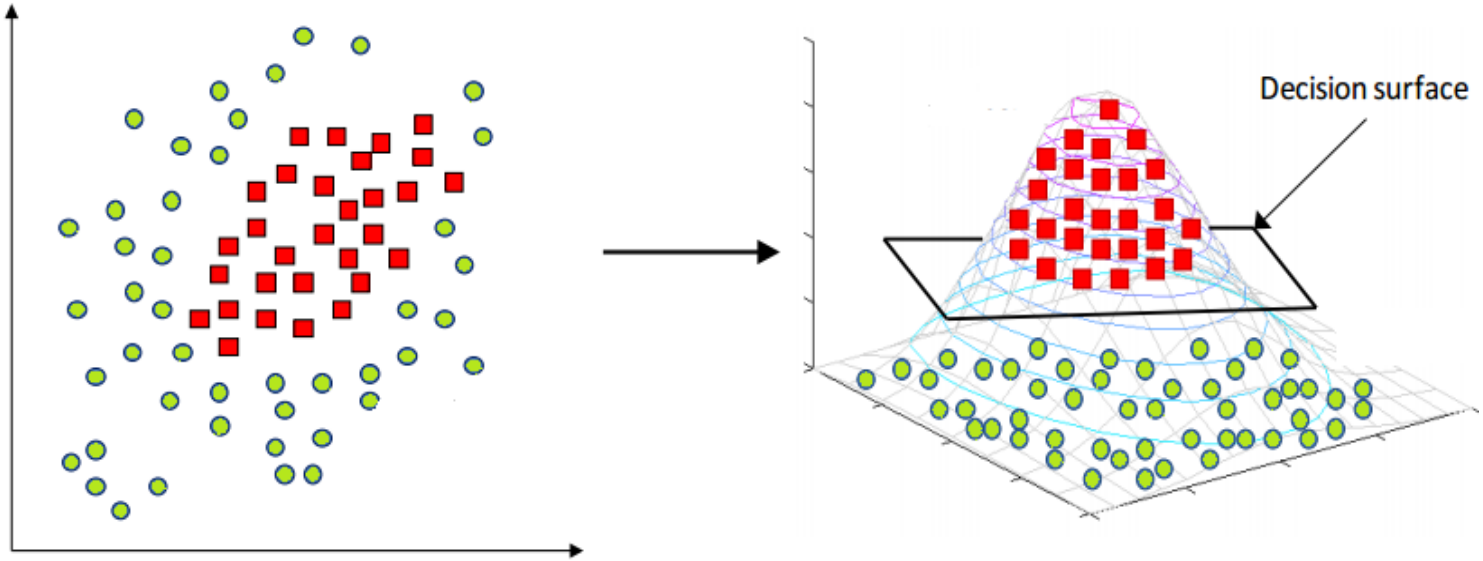
SVM on the Feature Space

- **The SVM decision boundary is:** $\mathbf{w}^{*\mathrm{T}}\boldsymbol{\phi}(\mathbf{x}) + b$

, which is

$$\sum_{i=1}^{N} \alpha_i y_i \boldsymbol{\phi}(\mathbf{x}_i)^{\mathrm{T}} \cdot \boldsymbol{\phi}(\mathbf{x}) + b$$





Kernel Trick

- **Observation:**
  - In the SVM process, the only operations on the feature space is **inner product** among two data vectors
  - Which means that we just need a scalar value computed from the feature space
- **Idea:**
  - There is a function that can calculate the inner product in feature space WITHOUT transforming data with $\boldsymbol{\Phi}$, but in the original dimension space $\boldsymbol{R}^{D}$.
- **Such functions are called the kernel functions**

Kernel Functions

- **If there exists a function Φ mapping $\mathbf{x} \in R^D$ to R^Q such that**

$$K(\mathbf{x}, \mathbf{x}') = \Phi(\mathbf{x})^{\mathrm{T}} \Phi(\mathbf{x}')$$

then, we call this $K(\mathbf{x}, \mathbf{x}')$ as a kernel function

- **Example:** $\mathbf{x} = (x_1, x_2)$

$$\mathbf{z} = \Phi(\mathbf{x}) = (\,1\,,\ x_1^2\,,\ x_2^2\,,\ \sqrt{2}x_1\,,\ \sqrt{2}x_2\,,\ \sqrt{2}\,x_1x_2\,)$$

$$K(\mathbf{x}, \mathbf{x}') = \mathbf{z}^{\mathsf{T}}\mathbf{z}' = 1 + x_1^2 x'^2_1 + x_2^2 x'^2_2 + 2x_1x'_1 + 2x_2x'_2 + 2x_1x'_1x_2x'_2$$

$$= (1 + x_1x'_1 + x_2x'_2)^2$$

$$= (1 + \mathbf{x}^{\mathsf{T}}\mathbf{x}')^2$$

Kernel Functions

- **Given $\mathbf{x}$ and $\mathbf{x}'$ in R^d,**

$$\mathbf{x} = (x_1, x_2, \dots, x_d)^T$$
$$\mathbf{x}' = (x_1', x_2', \dots, x_d')^T$$

- **Let a transformation function Φ that doubles the input dimension as follows ($\Phi: R^d \to R^{d^2}$):**

$$\Phi(\mathbf{x}) = \left(x_1^2, x_1x_2, \dots, x_1x_d, x_2x_1, x_2^2, \dots, x_2\,x_d, \dots, x_dx_1, x_dx_2, \dots, x_d^2\right)^T$$

- **Then, the inner product on the feature space will be:**

$$\boldsymbol{\Phi}(\mathbf{x})^{\boldsymbol{T}}\boldsymbol{\Phi}(\mathbf{x}') = \sum_{i,j=1}^{d}(x_ix_j)(x_i'x_j')$$

- **And, below is the kernel function that results in the same value!**

$$\sum_{i,j=1}^{d}(x_ix_j)(x_i'x_j') = \left(\sum_{i=1}^{d}(x_ix_i')\right)^{\mathbf{2}} = \left(\mathbf{x}^{\mathbf{T}}\mathbf{x}'\right)^{\mathbf{2}} \quad = \quad \boldsymbol{k}(\mathbf{x}, \mathbf{x}')!$$

Kernel Functions

- **Example: given $\mathbf{x}$ and $\mathbf{y}$ in R^3,**

$$\mathbf{x} = (x_1, x_2, x_3)^T$$

$$\mathbf{y} = (y_1, y_2, y_3)^T$$

- **The transformation function Φ is defined as:**

$$\Phi(\mathbf{x}) = (x_1^2, x_1x_2, x_1x_3, x_2x_1, x_2^2, x_2x_3, x_3x_1, x_3x_2, x_3^2)^T$$

- **Then, the inner product on the feature space will be:**

$$\boldsymbol{\Phi}(\mathbf{x})^{\boldsymbol{T}}\boldsymbol{\Phi}(\mathbf{y}) = \sum_{i,j=1}^{3} x_i x_j y_i y_j = x_1x_1y_1y_1 + \cdots + x_3x_3y_3y_3$$

- **There is a kernel function that results in the same value!**

$$\boldsymbol{k}(\mathbf{x}, \mathbf{y}) = \left(\mathbf{x}^{\boldsymbol{T}}\mathbf{y}\right)^{\mathbf{2}} = (x_1y_1 + x_2y_2 + x_3y_3)^2$$

Kernel Function Examples

- **Recommended kernels**
  - Polynomial kernel $$K(x,y) = (x \cdot y + c)^d, \quad c > 0$$
  - RBF (radial-basis functions) kernel $$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma ||\mathbf{x} - \mathbf{x}'||^2)$$
  - Gaussian kernel $$K(x,y) = \exp\left(-\frac{||x-y||^2}{2\sigma^2}\right), \quad \sigma \neq 0$$
  - Sigmoid kernel $$K(x,y) = \tanh(a(x \cdot y) + b), \quad a, b \geq 0$$
  - ....

RBF Kernel

*Taylor expansion: $e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots,$

- **Using Taylor expansion, (**set $\gamma = 1$ and 1-dimension for simplicity**)**

$$e^{-(x_i - x_j)^2} = e^{\left(-x_i^2 - x_j^2\right)} \cdot e^{2x_i x_j} = e^{\left(-x_i^2 - x_j^2\right)} \left(1 + \frac{2x_i x_j}{1!} + \frac{(2x_i x_j)^2}{2!} + \cdots\right)$$

$$= e^{-x_i^2} \cdot e^{-x_j^2} \left(1 \cdot 1 + \sqrt{\frac{2}{1!}} x_i \cdot \sqrt{\frac{2}{1!}} x_j + \sqrt{\frac{2^2}{2!}} (x_i)^2 \cdot \sqrt{\frac{2^2}{2!}} (x_j)^2 \cdots\right)$$

$$= \phi(x_i)^T \phi(x_j)$$

where, $\phi(x) = e^{-x^2} \left(1, \sqrt{\frac{2}{1!}} x, \sqrt{\frac{2^2}{2!}} x^2, \cdots, \sqrt{\frac{2^\infty}{\infty!}} x^\infty\right)$

*It transforms data x to an infinite dimension space!!!

Kernel Trick Summary

- **Kernel: "inner product in feature space"**
  - Given two data $\mathbf{x}$ and $\mathbf{x}'$, and a mapping function $\mathbf{\Phi}$, we define the corresponding kernel as:

$$K(\mathbf{x}, \mathbf{x}') = \mathbf{\Phi}(\mathbf{x})^{\mathbf{T}}\mathbf{\Phi}(\mathbf{x}') = \mathbf{z}^{\mathbf{T}}\mathbf{z}'$$

- **Trick: computing $K(\mathbf{x}, \mathbf{x}')$ without actual transformation**
  - Computing the kernel => mapping two data, then taking inner product
  - It may be computationally expensive
  - But in some situation, **we can compute $K(\mathbf{x}, \mathbf{x}')$ via computation from the input space data!**

Kernel Trick Applied to SVM

- **We can put the kernel function to the SVM formulations**
  - In the dual form:

$$\max \quad L_D = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \boxed{\Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j)}$$

$$\Downarrow$$

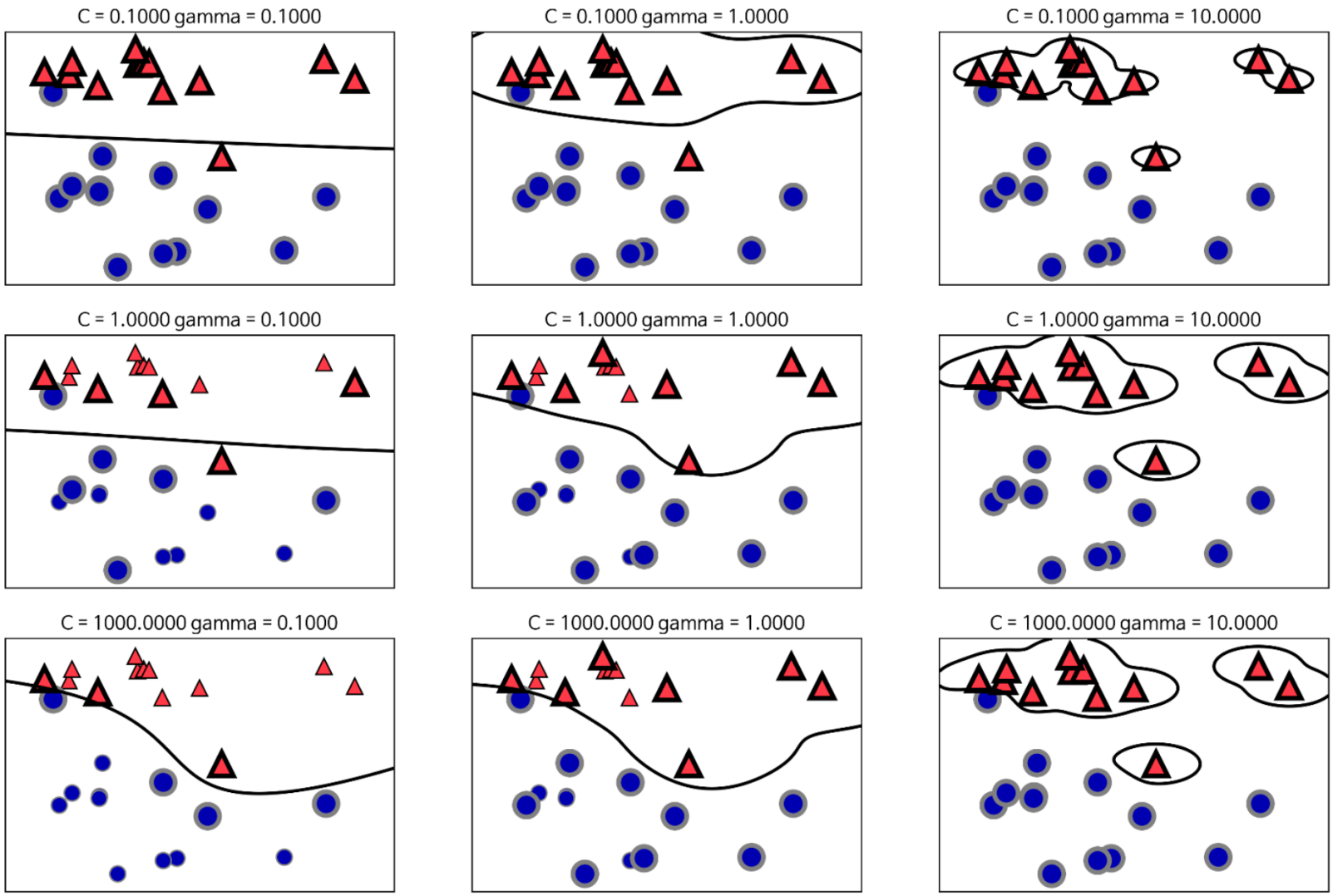
$$\max \quad L_D = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \boxed{K(\mathbf{x}_i, \mathbf{x}_j)}$$

  - Testing a new data:

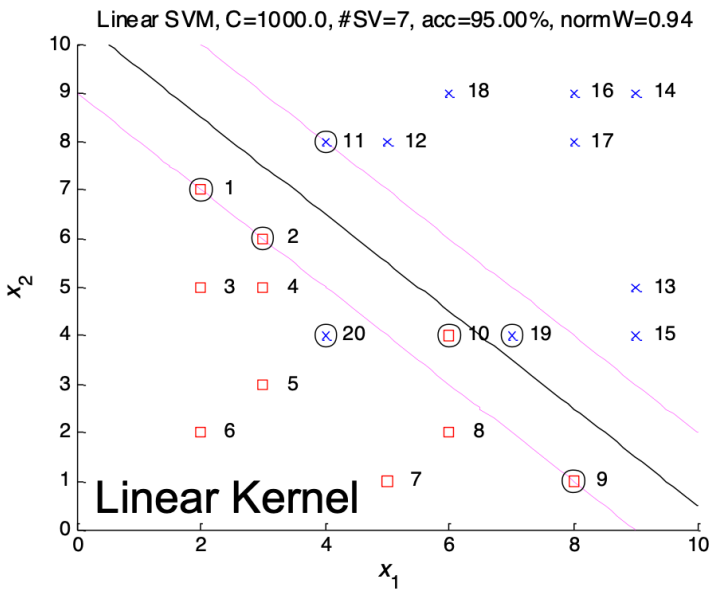
$$\sum_{\boldsymbol{i=1}}^{\boldsymbol{N}} \alpha_i y_i \boldsymbol{\phi}(\mathbf{x}_i)^{\mathrm{T}} \cdot \boldsymbol{\phi}(\mathbf{x}) + b \quad \Rightarrow \quad \sum_{i=1}^{N} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$

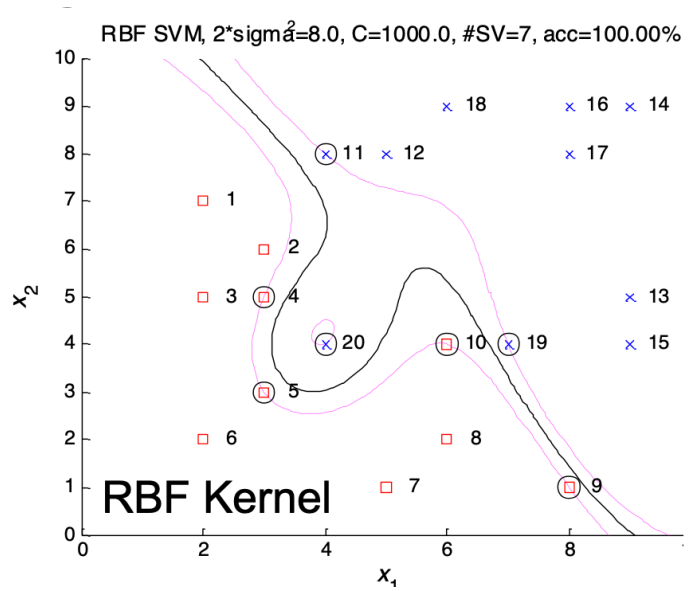
Results

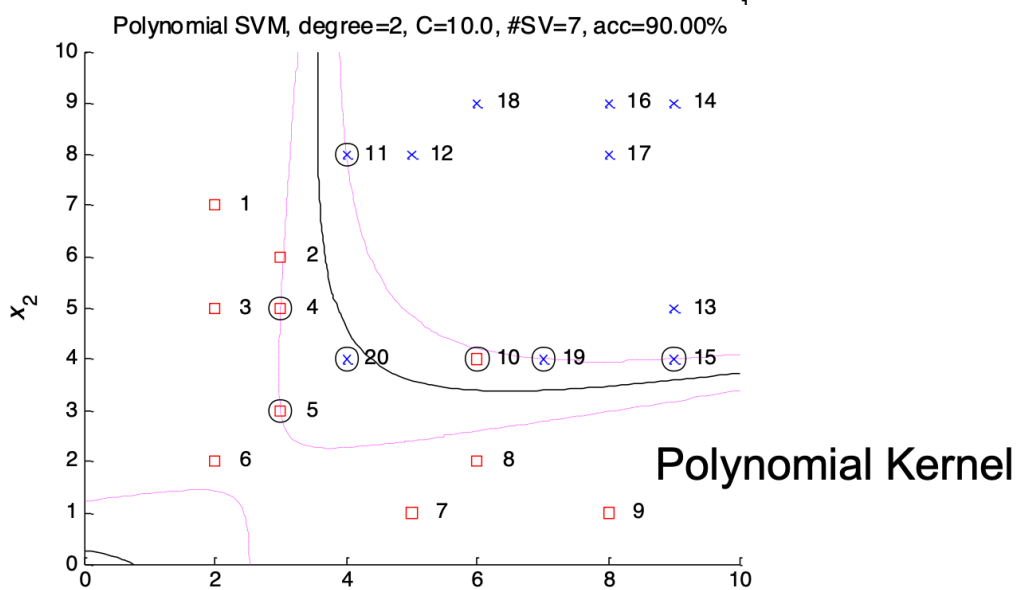
RBF Kernel is used here



Results with Different Kernels







Thank You